EKS Services Health Check & Validation Report

1. Objective

Project: EKS Services Health Check Validation

Objective:
Validate the health and availability of all application services running on Amazon EKS, identify common Kubernetes/infra issues, troubleshoot failures, and document the validation results with evidence.

Validation areas

- Pod health
- Node health
- Deployment / ReplicaSet
- Service
- Ingress / Load Balancer
- ConfigMap / Secret
- Persistent Volume / Storage
- Container image
- ImagePullSecret / ECR
- CPU / Memory
- Liveness / Readiness / Startup probes

- DNS
- Network connectivity
- NetworkPolicy / Security Groups
- HPA
- Cluster Autoscaler
- Events and logs
- Application health endpoints
- Resource limits/requests
- Kubernetes scheduling
- Service-to-service communication

---

2. Overall Health Check

Start with these commands for every EKS cluster.

```
kubectl get nodes
kubectl get pods -A
kubectl get deployments -A
kubectl get services -A
kubectl get ingress -A
kubectl get pvc -A
kubectl get pv
kubectl get events -A --sort-by=.lastTimestamp
```

Check cluster context:

```
kubectl config current-context
kubectl cluster-info
```

Check node resources:

```
kubectl top nodes
kubectl top pods -A
```

Expected result

Validation	Expected
Nodes	Ready
Pods	Running/Completed
Deployments	Available replicas = Desired replicas

Validation	Expected
Services	Correct endpoints
PVC	Bound
Ingress	Correct address
CPU/Memory	Within limits
Events	No abnormal warnings
Probes	Passing
Images	Successfully pulled
Networking	Application communication working

---

3. Pod Failures

Issue

Pod is not running or repeatedly restarting.

Common statuses:

```
Pending
CrashLoopBackOff
Error
ImagePullBackOff
ErrImagePull
OOMKilled
ContainerCreating
Terminating
```

Troubleshooting

```
kubectl get pods -n <namespace>
```

Check detailed information:

```
kubectl describe pod <pod-name> -n <namespace>
```

Check logs:

```
kubectl logs <pod-name> -n <namespace>
```

Previous container logs:

```
kubectl logs <pod-name> -n <namespace> --previous
```

Check events:

```
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

Validate

Confirm:

```
kubectl get pod <pod-name> -n <namespace>
```

Expected:

```
STATUS: Running
READY: 1/1
RESTARTS: 0 or within acceptable range
```

---

4. CrashLoopBackOff

Issue

Container starts but repeatedly crashes.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --previous
```

Check:

- Application startup error
- Wrong environment variable
- Missing Secret
- Missing ConfigMap
- Database connectivity
- Incorrect command/entrypoint
- Application port
- Memory issue
- Dependency failure

Validate

Restart/redeploy after fixing the root cause:

```
kubectl rollout restart deployment <deployment> -n <namespace>
```

Then:

```
kubectl rollout status deployment <deployment> -n <namespace>
kubectl get pods -n <namespace>
```

---

5. Pod Pending

Issue

Pod cannot be scheduled onto a node.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
```

Look at **Events**.

Common reasons:

- Insufficient CPU
- Insufficient memory
- NodeSelector mismatch
- Taints/tolerations
- Affinity rules
- No available nodes
- PVC not available

Check:

```
kubectl get nodes
kubectl describe nodes
kubectl top nodes
```

Validate

Pod should move:

```
Pending → ContainerCreating → Running
```

---

6. Node NotReady

Issue

EKS worker node is unhealthy or unavailable.

Troubleshooting

```
kubectl get nodes
kubectl describe node <node-name>
```

Check:

- CPU
- Memory
- Disk pressure
- Network unavailable
- Kubelet status
- Node conditions

```
kubectl top node <node-name>
```

Check node conditions:

```
kubectl describe node <node-name>
```

Validate

Expected:

```
STATUS: Ready
```

Also ensure workloads are successfully scheduled.

---

7. High CPU / Memory

Issue

Node or pod is consuming excessive resources.

Troubleshooting

```
kubectl top nodes
kubectl top pods -A
```

Check deployment:

```
kubectl describe deployment <deployment> -n <namespace>
```

Check requests/limits:

```
kubectl get pod <pod-name> -o yaml
```

Common causes

- Memory leak
- High traffic
- Incorrect resource limits
- Application issue
- Insufficient node capacity

Validate

CPU and memory should return to an acceptable range after remediation.

---

8. OOMKilled

Issue

Container exceeded its memory limit.

Check:

```
kubectl describe pod <pod-name> -n <namespace>
```

Look for:

```
Reason: OOMKilled
```

Troubleshooting

Check:

```
kubectl top pod <pod-name> -n <namespace>
```

Review:

- Memory usage
- Memory limit
- Application logs
- Traffic level

Validate

After correction:

```
kubectl get pod <pod-name> -n <namespace>
```

Confirm no continuous restarts and no new OOMKilled events.

---

9. ImagePullBackOff / ErrImagePull

Issue

Kubernetes cannot pull the container image.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
```

Check image:

```
kubectl get deployment <deployment> -n <namespace> -o yaml
```

Common reasons:

- Incorrect image name
- Incorrect image tag
- Image doesn't exist
- ECR authentication problem
- IAM permission issue
- Network connectivity
- ImagePullSecret issue

For ECR:

```
aws ecr describe-images \
  --repository-name <repository>
```

Validate

After fixing:

```
kubectl get pods -n <namespace>
```

Expected:

```
Running
```

---

10. Service / Endpoint Failure

Issue

Pod is running but application is not reachable through Kubernetes Service.

Troubleshooting

```
kubectl get svc -n <namespace>
kubectl describe svc <service> -n <namespace>
```

Check endpoints:

```
kubectl get endpoints <service> -n <namespace>
```

Or:

```
kubectl get endpointslices -n <namespace>
```

Check Service selector:

```
kubectl get svc <service> -n <namespace> -o yaml
```

Compare with pod labels:

```
kubectl get pods -n <namespace> --show-labels
```

Common issue

Service selector doesn't match pod labels.

Validate

Service should have healthy endpoints.

---

11. Readiness Probe Failure

Issue

Application is running but Kubernetes doesn't consider it ready.

Example:

```
Readiness probe failed
```

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
```

Check:

- Probe URL
- Port
- Path
- Application startup time
- Network connectivity
- Application health endpoint

Test from inside pod:

```
kubectl exec -it <pod-name> -n <namespace> -- curl localhost:<port>/<health-path>
```

Validate

Expected:

```
READY: 1/1
```

And Service should have endpoints.

---

12. Liveness Probe Failure

Issue

Kubernetes thinks the application is unhealthy and restarts the container.

Troubleshooting

```
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
```

Check:

- Wrong health endpoint
- Incorrect port
- Application startup time
- Timeout
- Initial delay
- Application dependency failure

Validate

Confirm:

```
kubectl get pods -n <namespace>
```

Restart count should stop increasing.

---

13. Startup Probe Failure

Issue

Application takes longer to start than the configured startup probe allows.

Troubleshooting

Check:

```
kubectl describe pod <pod-name> -n <namespace>
```

Review:

```
initialDelaySeconds
periodSeconds
failureThreshold
timeoutSeconds
```

Validate

Application should successfully start and become:

```
Ready
```

---

14. ConfigMap Issue

Issue

Application is missing or receiving incorrect configuration.

Troubleshooting

```
kubectl get configmap -n <namespace>
kubectl describe configmap <configmap-name> -n <namespace>
```

Check deployment:

```
kubectl describe deployment <deployment> -n <namespace>
```

Verify environment variables:

```
kubectl exec <pod-name> -n <namespace> -- env
```

Validate

Application starts successfully with the expected configuration.

---

15. Secret Issue

Issue

Application cannot access credentials/configuration stored in Kubernetes Secret.

Check:

```
kubectl get secrets -n <namespace>
kubectl describe secret <secret-name> -n <namespace>
```

Check pod:

```
kubectl describe pod <pod-name> -n <namespace>
```

Common causes:

- Secret doesn't exist
- Wrong Secret name
- Incorrect key
- Secret not mounted
- IAM/External Secrets issue

Validate

Application should start and successfully authenticate to the required dependency.

---

16. PVC / Storage Failure

Issue

Pod cannot mount persistent storage.

Check:

```
kubectl get pvc -A
kubectl get pv
```

Detailed:

```
kubectl describe pvc <pvc-name> -n <namespace>
```

Check:

```
kubectl get storageclass
```

Common issues

- PVC Pending
- StorageClass issue
- EBS volume problem
- Availability Zone mismatch
- CSI driver issue
- Volume attachment failure
- Insufficient permissions

Validate

Expected:

```
PVC: Bound
Pod: Running
Volume: Mounted
```

---

17. EBS CSI Driver Failure

Issue

EKS cannot provision or attach an EBS volume.

Check:

```
kubectl get pods -n kube-system | grep ebs
```

Check:

```
kubectl get csidrivers
```

Troubleshoot:

```
kubectl describe pod <pod-name> -n <namespace>
kubectl get events -A --sort-by=.lastTimestamp
```

Validate

PVC should become:

`Bound`

and the application pod should successfully mount the volume.

---

18. DNS Failure

Issue

Pod cannot resolve Kubernetes service or external hostname.

Test:

```
kubectl exec -it <pod-name> -n <namespace> -- nslookup kubernetes.default
```

Test service:

```
kubectl exec -it <pod-name> -n <namespace> -- nslookup <service-name>
```

Check CoreDNS:

```
kubectl get pods -n kube-system | grep coredns
```

Validate

DNS resolution should work from the application pod.

---

19. Network Connectivity Failure

Issue

Pod cannot communicate with another service or external system.

Troubleshooting

From pod:

```
kubectl exec -it <pod-name> -n <namespace> -- curl -v http://<service>:<port>
```

Check:

- Service
- Endpoints
- NetworkPolicy
- Security Groups
- NACL
- Route tables
- DNS
- Application port

Validate

Confirm successful connection and expected HTTP response.

---

20. Ingress / Load Balancer Failure

Issue

Application is working internally but unavailable externally.

Check:

```
kubectl get ingress -A
kubectl describe ingress <ingress-name> -n <namespace>
```

Check service:

```
kubectl get svc -n <namespace>
```

Check AWS Load Balancer/controller:

```
kubectl get pods -n kube-system
```

Validate:

- DNS
- Listener
- Target group
- Health checks
- Security groups
- Backend service
- Ingress rules

Validate

Test:

```
curl -I https://<application-url>
```

Expected:

```
HTTP 200/expected application response
```

---

21. Deployment / Replica Failure

Issue

Deployment doesn't have the expected number of available replicas.

```
kubectl get deployment -A
```

Check:

```
kubectl describe deployment <deployment> -n <namespace>
```

Check rollout:

```
kubectl rollout status deployment <deployment> -n <namespace>
```

Validate

Expected:

```
DESIRED = CURRENT = READY = AVAILABLE
```

---

22. Failed Deployment / Rollout

Issue

New application version is not successfully deployed.

Check:

```
kubectl rollout status deployment <deployment> -n <namespace>
```

History:

```
kubectl rollout history deployment <deployment> -n <namespace>
```

If required, rollback:

```
kubectl rollout undo deployment <deployment> -n <namespace>
```

Validate

```
kubectl get pods -n <namespace>
kubectl rollout status deployment <deployment> -n <namespace>
```

---

23. HPA Failure

Issue

Horizontal Pod Autoscaler isn't scaling as expected.

Check:

```
kubectl get hpa -A
kubectl describe hpa <hpa-name> -n <namespace>
```

Check metrics:

```
kubectl top pods -n <namespace>
```

Common issues:

- Metrics Server
- Incorrect CPU/memory requests
- Wrong target
- Application not generating load

Validate

Generate/test load if appropriate and verify replicas scale according to the configured policy.

---

24. Node Disk Pressure

Issue

Node has insufficient disk space.

Check:

```
kubectl describe node <node-name>
```

Look for:

```
DiskPressure=True
```

Check:

```
kubectl top nodes
```

Investigate:

- Container images
- Container logs
- Temporary files
- Unused images

Validate

Expected:

```
DiskPressure=False
```